PRAKTIKUM ALGORITMA DAN STRUKTUR DATA
JOBSHEET 10 QUEUE

NAMA : LUCKY SATRIA UTAMA
NIM : 254107060142
JURUSAN : TEKNOLOGI INFORMASI
PRODI : D-IV SISTEM INFORMASI BISNIS
KELAS : SIB – 1F

TAHUN 2025/2026

PERCOBAAN 1:

Queue.java

```
public class Queue {
    int[] data;
    int front;
    int rear;
    int size;
    int max;

    public Queue(int n) {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }

    public boolean IsFull() {
        if (size == max) {
            return true;
        } else {
            return false;
        }
    }


    public void peek() {
        if (!IsEmpty()) {
            System.out.println("Elemen terdepan: " + data[front]);
        } else {
            System.out.println("Queue masih kosong");
        }
    }

    public void print() {
        if (IsEmpty()) {
            System.out.println("Queue masih kosong");
        } else {
            int i = front;
            while (i != rear) {
                System.out.print(data[i] + " ");
```

```
                i = (i + 1) % max;
            }
            System.out.println("Jumlah elemen = " + size);
        }
    }

    public void clear() {
        if (!IsEmpty()) {
            front = rear = -1;
            size = 0;
            System.out.println("Queue berhasil dikosongkan");
        } else {
            System.out.println("Queue masih kosong");
        }
    }

    public void Enqueue(int dt) {
        if (IsFull()) {
            System.out.println("Queue sudah penuh");
        } else {
            if (IsEmpty()) {
                front = rear = 0;
            } else {
                if (rear == max - 1) {
                    rear = 0;
                } else {
                    rear++;
                }
            }
            data[rear] = dt;
            size++;
        }
    }

    public int Dequeue() {
        int dt = 0;
        if (IsEmpty()) {
            System.out.println("Queue masih kosong");
        } else {
            dt = data[front];
            size--;
            if (IsEmpty()) {
                front = rear = -1;
            } else {
                if (front == max - 1) {
                    front = 0;
                } else {
                    front++;
```

```
                }
            }
        }
        return dt;
    }
}
```

QueueMain.java

```
import java.util.Scanner;

public class QueueMain {

    public static void menu() {
        System.out.println("Masukkan operasi yang diinginkan: ");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Print");
        System.out.println("4. Peek");
        System.out.println("5. Clear");
        System.out.println("----------------");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan kapasitas queue: ");
        int n = sc.nextInt();
        Queue Q = new Queue(n);
        int pilih;

        do {
            menu();
            pilih = sc.nextInt();
            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
                    int data = sc.nextInt();
                    Q.Enqueue(data);
                    break;
                case 2:
                    int dataKeluar = Q.Dequeue();
                    if (dataKeluar != 0) {
                        System.out.println("Data yang dikeluarkan: " +
dataKeluar);
                        break;
                    }
                case 3:
                    Q.print();
```

```
                    break;
                case 4:
                    Q.peek();
                    break;
                case 5:
                    Q.clear();
                    break;
            }
        } while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih
== 5);
    }
}
```

Jawaban Pertanyaan Percobaan 1 - Queue

1. Mengapa nilai awal front dan rear bernilai -1, sementara size bernilai 0?

Nilai front = -1 dan rear = -1 digunakan sebagai penanda bahwa queue **masih kosong** (belum ada elemen). Nilai -1 dipilih karena indeks array dimulai dari 0, sehingga -1 bukan indeks yang valid dan menjadi tanda kondisi awal.

Sementara size = 0 karena memang belum ada elemen yang tersimpan di dalam queue. Atribut size menyimpan jumlah elemen aktual, sehingga logis dimulai dari 0.

2. Penjelasan potongan kode pada method Enqueue

Potongan kode yang dimaksud:

```
if (isEmpty()) { front = 0; }
rear = (rear + 1) % max;
data[rear] = dt;
size++;
```

Penjelasan:

1. Jika queue kosong (pertama kali diisi), front diset ke 0 agar menunjuk elemen pertama.
2. rear = (rear + 1) % max menggunakan **modulo** agar rear berputar kembali ke indeks 0 jika sudah mencapai akhir array (circular queue), sehingga slot yang dikosongkan bisa digunakan kembali.
3. data[rear] = dt menyimpan data baru di posisi rear.
4. size++ menambah jumlah elemen dalam queue.

3. Penjelasan potongan kode pada method Dequeue

Potongan kode yang dimaksud:

```
int temp = data[front];
front = (front + 1) % max;
size--;
if (size == 0) { front = -1; rear = -1; }
return temp;
```

Penjelasan:

5. data[front] mengambil elemen paling depan untuk dikembalikan.

6. front = (front + 1) % max memajukan posisi front ke elemen berikutnya dengan modulo untuk mendukung circular queue.
7. size-- mengurangi jumlah elemen.
8. Jika setelah dequeue size == 0, maka front dan rear direset ke -1 untuk menandai queue kembali kosong.

4. Mengapa variabel i pada method print dimulai dari front, bukan dari 0?

Karena pada **circular queue**, elemen pertama tidak selalu berada di indeks 0. Posisi elemen terdepan ditunjukkan oleh front. Jika iterasi dimulai dari 0, bisa mencetak slot array yang sudah kosong atau sudah di-dequeue. Dengan memulai dari i = front, data yang dicetak selalu dimulai dari elemen yang paling pertama masuk dan masih ada.

5. Penjelasan potongan kode pada method print

Potongan kode yang dimaksud:

```
for (int i = front; i != rear; i = (i + 1) % max) {
    System.out.println(data[i]);
}
System.out.println(data[rear]);
```

Perulangan berjalan dari front ke rear mengikuti arah antrian. Ekspresi i = (i + 1) % max memastikan indeks berputar secara circular, sehingga jika rear berada di indeks yang lebih kecil dari front (akibat putaran), elemen tetap dapat dicetak dengan benar.

6. Potongan kode yang merupakan Queue Overflow

Queue overflow terjadi saat enqueue dipanggil ketika queue sudah penuh. Potongan kodenya ada di method Enqueue:

```
if (isFull()) {
    System.out.println("Queue Overflow! Queue sudah penuh.");
    return;
}
```

Kondisi `isFull()` bernilai `true` ketika `size == max`, artinya semua slot array sudah terisi.

7. Modifikasi: Program berhenti saat Queue Overflow / Underflow

Menggunakan System.exit(1):

```
// Pada method Enqueue
if (isFull()) {
    System.out.println("Queue Overflow! Program dihentikan.");
    System.exit(1);
}
// Pada method Dequeue
if (isEmpty()) {
    System.out.println("Queue Underflow! Program dihentikan.");
    System.exit(1);
}
```

Atau menggunakan Exception (lebih profesional):

```
// Pada method Enqueue
if (isFull()) {
    throw new RuntimeException("Queue Overflow! Queue sudah penuh.");
}
```

```
// Pada method Dequeue
if (isEmpty()) {
    throw new RuntimeException("Queue Underflow! Queue kosong.");
}
```

Dengan melempar RuntimeException, program akan berhenti secara otomatis dan menampilkan pesan error beserta stack trace-nya.

PERCOBAAN 2:

Mahasiswa.java

```
public class Mahasiswa {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println("NIM   : " + nim);
        System.out.println("Nama  : " + nama);
        System.out.println("Prodi : " + prodi);
        System.out.println("Kelas : " + kelas);
    }
}
```

AntrianLayanan.java

```
public class AntrianLayanan {
    Mahasiswa[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan(int n) {
        max   = n;
        data  = new Mahasiswa[max];
        front = -1;
        rear  = -1;
        size  = 0;
```

```
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == max;
    }

    public void enqueue(Mahasiswa dt) {
        if (isFull()) {
            System.out.println("Antrian penuh! Tidak dapat menambahkan
data.");
        } else {
            if (isEmpty()) {
                front = 0;
            }
            rear = (rear + 1) % max;
            data[rear] = dt;
            size++;
            System.out.println("Mahasiswa " + dt.nama + " berhasil ditambahkan
ke antrian.");
        }
    }

    public Mahasiswa dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong! Tidak ada data yang bisa
dikeluarkan.");
            return null;
        } else {
            Mahasiswa mhs = data[front];
            data[front] = null;
            front = (front + 1) % max;
            size--;
            if (isEmpty()) {
                front = -1;
                rear  = -1;
            }
            return mhs;
        }
    }

    public void peek() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
        } else {
```

```
            System.out.println("=== Antrian Paling Depan ===");
            data[front].tampilkanData();
        }
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("=== Daftar Antrian Layanan ===");
            int i = front;
            for (int k = 0; k < size; k++) {
                System.out.println("--- Antrian ke-" + (k + 1) + " ---");
                data[i].tampilkanData();
                i = (i + 1) % max;
            }
        }
    }

    public void clear() {
        for (int i = 0; i < max; i++) {
            data[i] = null;
        }
        front = -1;
        rear  = -1;
        size  = 0;
        System.out.println("Semua antrian berhasil dihapus.");
    }

    public int getJumlahAntrian() {
        return size;
    }

    public void lihatAkhir() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("=== Antrian Paling Belakang ===");
            data[rear].tampilkanData();
        }
    }
}
```

LayananAkademikSIAKAD.java

```
import java.util.Scanner;
```

```java
public class LayananAkademikSIAKAD {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        AntrianLayanan antrian = new AntrianLayanan(5);

        int pilihan;

        do {
            System.out.println("\n==============================");
            System.out.println("  Layanan Akademik SIAKAD");
            System.out.println("==============================");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Panggil Antrian");
            System.out.println("3. Cek Antrian Paling Depan");
            System.out.println("4. Tampilkan Semua Antrian");
            System.out.println("5. Hapus Semua Antrian");
            System.out.println("6. Cek Antrian Paling Belakang");
            System.out.println("0. Keluar");
            System.out.print("Pilihan : ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.println("\n-- Tambah Antrian --");
                    System.out.print("NIM   : ");
                    String nim   = sc.nextLine();
                    System.out.print("Nama  : ");
                    String nama  = sc.nextLine();
                    System.out.print("Prodi : ");
                    String prodi = sc.nextLine();
                    System.out.print("Kelas : ");
                    String kelas = sc.nextLine();
                    Mahasiswa mhs = new Mahasiswa(nim, nama, prodi, kelas);
                    antrian.enqueue(mhs);
                    System.out.println("Jumlah antrian saat ini : " +
antrian.getJumlahAntrian());
                    break;

                case 2:
                    System.out.println("\n-- Panggil Antrian --");
                    Mahasiswa dipanggil = antrian.dequeue();
                    if (dipanggil != null) {
                        System.out.println("Mahasiswa berikut dipanggil untuk
layanan:");
```

```
                        dipanggil.tampilkanData();
                        System.out.println("Jumlah antrian tersisa : " +
antrian.getJumlahAntrian());
                    }
                    break;

                case 3:
                    System.out.println();
                    antrian.peek();
                    break;

                case 4:
                    System.out.println();
                    antrian.print();
                    break;

                case 5:
                    System.out.println();
                    antrian.clear();
                    break;

                case 6:
                    System.out.println();
                    antrian.lihatAkhir();
                    break;

                case 0:
                    System.out.println("Terima kasih. Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan != 0);

        sc.close();
    }
}
```

Pertanyaan 2.2.3

Soal: Lakukan modifikasi program dengan menambahkan method baru bernama ***LihatAkhir*** pada class ***AntrianLayanan*** yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu **6. Cek Antrian paling belakang** pada class ***LayananAkademikSIAKAD*** sehingga method ***LihatAkhir*** dapat dipanggil!

Jawaban:

A. Konsep Method **lihatAkhir()**

Pada struktur data Queue, elemen dikelola berdasarkan prinsip FIFO (First In, First Out). Atribut rear menyimpan indeks posisi elemen paling belakang dalam antrian. Method lihatAkhir() memanfaatkan atribut rear tersebut untuk langsung mengakses dan menampilkan data mahasiswa yang berada di posisi paling belakang antrian tanpa mengeluarkannya (non-destructive).

Sebelum menampilkan data, method ini terlebih dahulu memeriksa apakah antrian dalam kondisi kosong menggunakan method isEmpty(). Jika antrian kosong (front == -1 dan rear == -1), maka program akan menampilkan pesan "Antrian kosong!" dan tidak melakukan operasi apapun pada array data[].

B. Penambahan Method **lihatAkhir()** pada Class **AntrianLayanan**

Berikut adalah potongan kode method lihatAkhir() yang ditambahkan ke dalam class AntrianLayanan:

```
// Method untuk melihat antrian paling belakang
public void lihatAkhir() {
    if (isEmpty()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("=== Antrian Paling Belakang ===");
        data[rear].tampilkanData();
    }
}
```

Penjelasan baris per baris pada method lihatAkhir():

1) if (isEmpty()) – Memeriksa apakah antrian kosong. Jika kosong maka atribut rear bernilai -1 sehingga data[rear] akan mengakibatkan ArrayIndexOutOfBoundsException jika tidak dicegah.

2) data[rear].tampilkanData() – Mengakses elemen pada indeks rear dari array data[] (bertipe Mahasiswa[]) kemudian memanggil method tampilkanData() untuk menampilkan NIM, nama, prodi, dan kelas.

C. Penambahan Menu 6 pada Class **LayananAkademikSIAKAD**

Agar method lihatAkhir() dapat dipanggil oleh pengguna melalui program, ditambahkan pilihan menu 6 pada tampilan menu utama dan blok case 6 di dalam struktur switch-case pada method main:

Penambahan pada tampilan menu:

```
System.out.println("6. Cek Antrian Paling Belakang");
```

Penambahan pada blok switch-case:

```
case 6:
    antrian.lihatAkhir();
    break;
```

D. Contoh Output Program

Berikut adalah contoh output saat pengguna memilih menu 6 setelah menambahkan beberapa mahasiswa ke antrian:

```
==============================
  Layanan Akademik SIAKAD
==============================
1. Tambah Antrian
2. Panggil Antrian
3. Cek Antrian Paling Depan
4. Tampilkan Semua Antrian
5. Hapus Semua Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilihan : 6

=== Antrian Paling Belakang ===
NIM   : 2024003
Nama  : Ahmad Fauzi
Prodi : Teknik Informatika
Kelas : TI-1B
```

E. Kesimpulan

Method lihatAkhir() menggunakan atribut rear sebagai acuan indeks elemen terbaru yang masuk ke antrian. Berbeda dengan method peek() yang menampilkan elemen terdepan (front), lihatAkhir() menampilkan elemen paling belakang (rear). Keduanya bersifat non-destructive, artinya tidak mengubah isi maupun struktur antrian. Penambahan menu 6 pada LayananAkademikSIAKAD memungkinkan petugas layanan akademik untuk mengetahui mahasiswa mana yang terakhir mendaftar antrian tanpa harus menampilkan seluruh daftar antrian.

Tugas:
Mahasiswa.java

```
public class Mahasiswa {
    String nim, nama, prodi, kelas;

    public Mahasiswa(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " +
kelas);
    }
}
```

AntrianKRS.java

```
public class AntrianKRS {
    Mahasiswa[] data;
    int front, rear, size, max;
    int totalSudahKRS = 0;

    public AntrianKRS(int n) {
        max = n;
        data = new Mahasiswa[max];
        size = 0;
        front = 0;
        rear = -1;
    }

    public boolean isEmpty() { return size == 0; }
    public boolean isFull() { return size == max; }

    public void enqueue(Mahasiswa m) {
        if (!isFull()) {
            rear = (rear + 1) % max;
            data[rear] = m;
            size++;
        } else {
            System.out.println("Antrian sudah penuh!");
        }
    }

    public void panggilKRS() {
        if (size >= 2) {
```

```
            System.out.println("Memproses KRS untuk 2 mahasiswa:");
            for (int i = 0; i < 2; i++) {
                Mahasiswa m = data[front];
                System.out.print("Diproses: ");
                m.tampilkanData();
                front = (front + 1) % max;
                size--;
                totalSudahKRS++;
            }
        } else {
            System.out.println("Minimal harus ada 2 mahasiswa untuk memulai
proses.");
        }
    }

    public void printSemua() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            int i = front;
            while (i != rear) {
                data[i].tampilkanData();
                i = (i + 1) % max;
            }
            data[i].tampilkanData();
        }
    }

    public void peekDuaTerdepan() {
        if (size >= 2) {
            System.out.println("2 Antrian Terdepan:");
            data[front].tampilkanData();
            data[(front + 1) % max].tampilkanData();
        } else {
            System.out.println("Data tidak cukup untuk menampilkan 2
terdepan.");
        }
    }

    public void peekTerakhir() {
        if (!isEmpty()) {
            System.out.print("Antrian Terakhir: ");
            data[rear].tampilkanData();
        }
    }

    public void cetakStatistik(int kuotaDPA) {
        System.out.println("Jumlah antrian saat ini: " + size);
```

```
        System.out.println("Total sudah proses KRS: " + totalSudahKRS);
        System.out.println("Sisa mahasiswa belum ditangani DPA: " + (kuotaDPA
- totalSudahKRS));
    }

    public void clear() {
        front = 0;
        rear = -1;
        size = 0;
        System.out.println("Antrian berhasil dikosongkan.");
    }
}
```

MainKRS.java

```
import java.util.Scanner;

public class MainKRS {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianKRS antrian = new AntrianKRS(10);
        int kuotaDPA = 30;
        int pilih;

        do {
            System.out.println("\n--- MENU ANTRIAN KRS ---");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Panggil Antrian (2 Mhs)");
            System.out.println("3. Tampilkan Semua Antrian");
            System.out.println("4. Cek 2 Terdepan & Terakhir");
            System.out.println("5. Cetak Statistik");
            System.out.println("6. Kosongkan Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print("NIM: "); String nim = sc.nextLine();
                    System.out.print("Nama: "); String nama = sc.nextLine();
                    System.out.print("Prodi: "); String prodi = sc.nextLine();
                    System.out.print("Kelas: "); String kelas = sc.nextLine();
                    antrian.enqueue(new Mahasiswa(nim, nama, prodi, kelas));
                    break;
                case 2:
                    antrian.panggilKRS();
                    break;
                case 3:
```

```
                    antrian.printSemua();
                    break;
                case 4:
                    antrian.peekDuaTerdepan();
                    antrian.peekTerakhir();
                    break;
                case 5:
                    antrian.cetakStatistik(kuotaDPA);
                    break;
                case 6:
                    antrian.clear();
                    break;
            }
        } while (pilih != 0);
    }
}
```